

Bu dersimizde ise Graph Teorisi'nin temel gezme algoritmalarından olan **BFS** Algoritmasından bahsedeceğiz.

Bir önceki yazımızda graph'ı gezmenin ne olduğundan bahsetmiştik ve DFS algoritmasını anlatmıştık. Şimdi ise lineer bir gezme algoritması olan **BFS** Algoritmasından bahsedeceğiz.

BFS (Breadth First Search) Nedir?

İsminin Türkçe'ye çevirmek istersek **Genişlik Öncelikli Arama** diyebiliriz. Bilmiyorum dikkatinizi çekti mi iki algoritmanın da son 2 kelimesi aynı "First Search" yani öncelikli arama anlamına geliyor. Aslında iki algoritma da aynı amaç için var, bir graph'ı gezmek. Ancak kullanacağımız alanlardaki farklılıklara uyum sağlayabilmesi için gezerken **belli öncelikler belirlemek** gerekiyor. Bu öncelik farklılıkları da graph gezme algoritmalarını oluşturuyor.

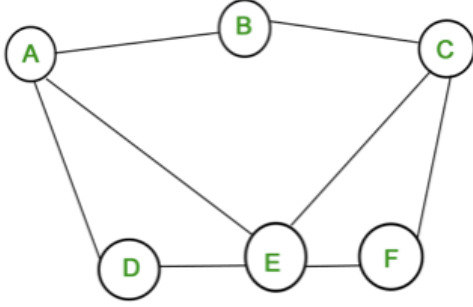
BFS Nasıl Çalışır?

Bu sefer önceliğimiz derinlik değil de genişlik olacak. Yani her bir node için aynı anda bütün **müsait çocuklarına** gideceğiz ve oradaki gezme işlemimiz bittiğinde aynı şekilde bu sefer gezdiğimiz node'ların çocuklarına aynı anda gideceğiz. İkidir "aynı anda" ifadesini kullanıyorum ancak malesef gerçek anlamda aynı anda gidemeyeceğiz. Çünkü algoritmanın **lineer** bir şekilde ilerlemesini istiyoruz. Bunu sağlamak için ise **queue** adında bir veri yapısı kullanacağız. Bu veri tipini basit biri dizi olarak düşünebilirsiniz ancak dizinin sadece o an en üstte bulunan elemanını silebiliyoruz ve diziyi sadece en aşağıdan veri ekleyebiliyoruz. Daha fazla ayrıntıya veri yapıları dersinde değinmenin daha uygun olacağını düşünüyorum.

Öncelikle başlangıç noktasını **queue'ya** ilk veri olarak ekliyoruz. Daha sonra queue dizisi boş olmadığı sürece aşağıdaki işlemleri yapıyoruz.

1. Eğer queue boş ise programı bitir. Değil ise queue'nun en üstündeki node'u al.
2. Bu node'un bütün çocuklarını döngü yardımıyla gez.
3. Daha önce gezilmemiş bütün çocuklarını queue'nun altına ekle.
4. Şu anki node'u gezme işlemi bittiği için queue'dan sil.
5. 1. Adıma geri dön.

BFS Örneği



İlk olarak B'yi queue'ya ekleyelim. B'nin gezilmesi müsait çocuklarını queue'ya ekliyoruz bunlar A ve C. B'yi gezme işlemimiz bittiği için queue'dan siliyoruz. Önce A'yı eklediğimiz için sıra A'ya geliyor. A'nın gezilebilir çocukları D ve E bunları da ekleyip A'yı siliyoruz. Şu an queue'nun içindeki sıralama C D E şeklinde. C'yi alıyoruz ve F'yi ekliyoruz. Daha sonra sırayla D, E ve F'yi geziyoruz. En son F'yi de gezdikten sonra siliyoruz ve queue'da veri kalmıyor. Bu nedenle program bitiyor ve bütün node'ları yollara uygun bir biçimde gezmiş oluyoruz.

BFS Karmaşıklığı

DFS'e benzer bir şekilde her bir node'a yalnızca 1 defa gittiğimiz için karmaşıklık $O(N)$ oluyor.

C++ Dilinde Kodlanması

```
int visited[100],graph[3][3]={1,1,1},{1,1,1},{1,1,1},n=3;
queue<int>que;

int startNode = 0;
que.push(startNode);

while(!que.empty()){
    int curNode = que.front();
    cout << curNode << " ";
    visited[curNode] = 1;

    for(int nextNode = 0; nextNode < n; nextNode++){
        if(graph[curNode][nextNode] == 1 and visited[nextNode] == 0){

            visited[nextNode] = 1;
            que.push(nextNode);
        }
    }
    que.pop();
}
```